

DOCUMENTAȚIE PROIECT POO

Sistem Integrat de Gestiune Depozit Echipamente IT

"Nexus Tech Store"

Disciplina: Programare Orientată pe Obiecte (POO)

Coordonator: Prodan Remus

Student: Cîrdeiu Loredana

Grupa: 3122B

1. Descrierea Generală a Aplicației

Proiectul reprezintă o soluție software completă de tip Enterprise Resource Planning (ERP) combinată cu un modul Point of Sale (POS), personalizată pentru un showroom modern de retail și componente hardware denumit generic **"Nexus Tech Store"**. Aplicația este optimizată pentru a gestiona inventare voluminoase (baze de date de peste 400 de articole hardware) și oferă operatorilor de la terminale o gamă largă de instrumente predictive și financiare.

Identificatorii unici ai produselor din structura de date sunt tratați conceptual drept coduri de bare EAN. Sistemul monitorizează stocurile de la raft în timp real, generează alerte automate la praguri critice, înregistrează tranzacțiile financiare prin șabloane de clase (Templates) și pune la dispoziție o interfață vizuală securizată bazată pe standarde moderne de codificare, utilizând paleta cromatică corporativă albastru-alb.

2. Specificații Funcționale

Sistemul acoperă următoarele fluxuri logistice și financiare esențiale din cadrul showroom-ului:

- **Verificare Stoc Critic (Opțiunea 1):** Scanează maparea de produse și extrage instantaneu componentele ale căror unități fizice au scăzut sub pragul minim de siguranță la raft.
- **Recomandări automate de Reaprovizionare (Opțiunea 2):** Algoritm care analizează automat deficitul și sugerează loturile optime ce trebuie comandate de la furnizori pentru a reface stocurile.
- **Modul Vânzare POS cu Nexus Card (Opțiunea 3):** Permite achiziția de hardware prin scanarea IDului și a cantității. Include un simulator modern de loialitate care recunoaște vouchere comerciale (**TECH10** , **STUDENT20** , **NEXUS30**), recalculează prețul clientului și emite un bon fiscal detaliat direct în consolă.
- **Recepție Loturi Noi (Opțiunea 4):** Permite înregistrarea intrărilor de marfă primite de la camioanele furnizorilor, crescând stocurile bazei de date.
- **Jurnal de Audit și Istoric (Opțiunea 5):** Afișează toate operațiunile efectuate în sesiune, stocate cronologic, grație claselor de tip șablon.

- **Căutare Avansată Case-Insensitive (Opțiunea 6):** Permite identificarea rapidă a oricărui echipament după un text parțial (ex: tastatură, mouse), ignorând diferențele de litere mari sau mici.
- **Panou de Indicatori Financieri (Opțiunea 7):** Calculează automat activele totale ale magazinului (valoarea mărfii în RON), numărul de produse active și reperele cele mai valoroase/ieftine din patrimoniu.
- **Export Managerial HTML (Opțiunea 8):** Generează automat un fișier web (`raport_inventar_NexusIT.html`) complet stilizat cu tabele moderne și insigne colorate (Badges) pentru a fi analizat direct în browser sau trimis către audit.

3. Arhitectura Software și Designul POO

Aplicația respectă principiile fundamentale ale Programării Orientate pe Obiecte și Clean Code:

Încapsularea Datelor

Toate atributele din clasele `Produs`, `Tranzactie` și `Depozit` sunt protejate prin modificatorul de acces `private`. Expunerea datelor se face controlat, exclusiv prin metode publice de tip Getter și Setter, garantând integritatea bazei de date.

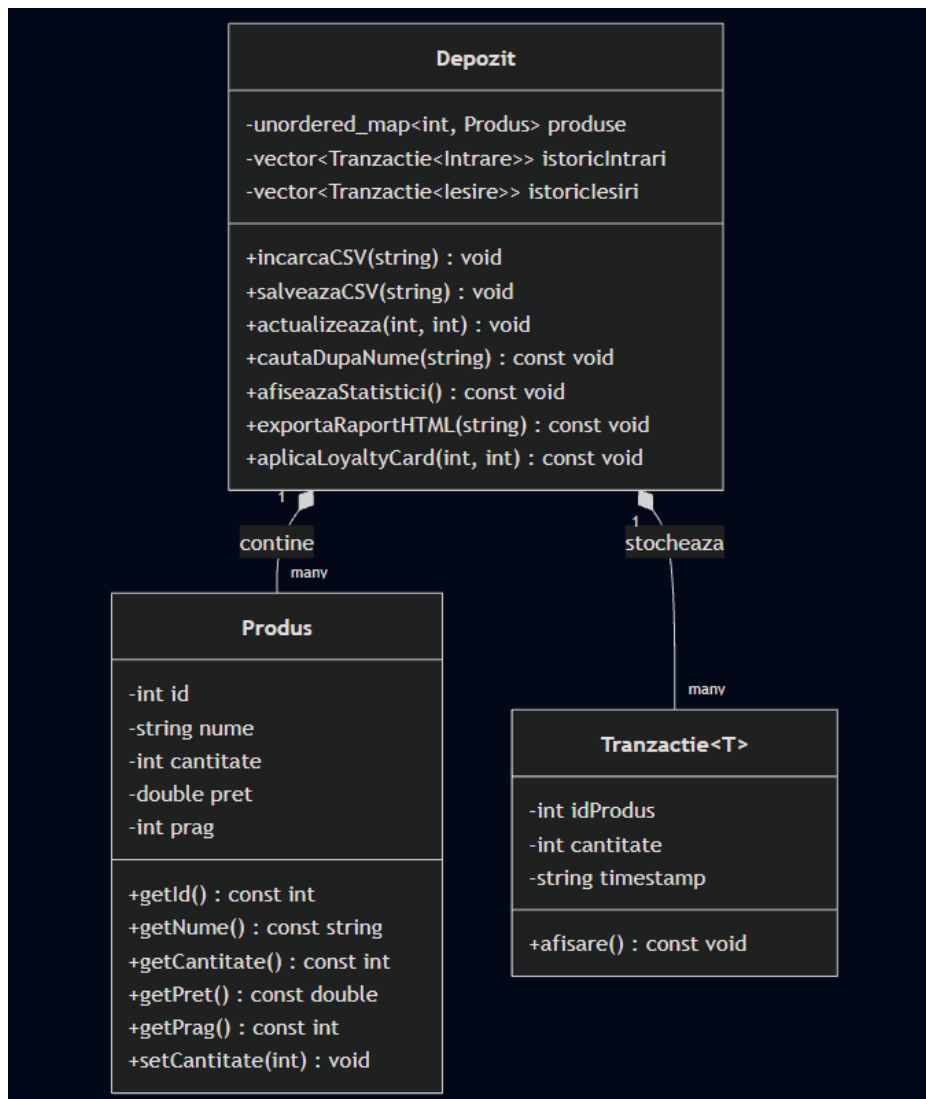
Programare Generică (Templates)

S-a implementat clasa șablon `Tranzactie<T>`, instanțiată pe baza structurilor logice `Intrare` și `Iesire`. Aceasta permite reutilizarea aceleiași structuri logice pentru fluxuri logistice total diferite, unificând comportamentul de jurnalizare.

Eficiența Structurilor de Date (STL)

Pentru produse s-a optat pentru `std::unordered_map<int, Produs>`. Această alegere asigură o complexitate de timp ideală de $O(1)$ (timp constant) pentru operațiunile de căutare, modificare și ștergere a reperelor pe baza ID-ului (codului de bare), eliminând complet latențele specifice vectorilor clasici $O(n)$ la volume mari de date.

4. Diagrama de Clase UML



5. Ghid de Implementare și Integrare Cod

Pentru a asigura funcționarea perfectă a noului modul POS și a sistemului de loialitate, codul din proiect se integrează conform instrucțiunilor de mai jos:

Antetul clasei (include/Depozit.h)

```
void aplicaLoyaltyCard(int id, int cantitate) const;
```

Definirea logicii de business (src/Depozit.cpp)

```
void Depozit::aplicaLoyaltyCard(int id, int cantitate) const {
    auto it = produse.find(id);
    if (it == produse.end()) {
        std::cout << "\033[31m[EROARE] Produsul nu exista!\033[0m\n";
        return;
    }
    const Produs& p = it->second;

    std::string codPromo;
    std::cout << "\033[34m[POS] Scanati Nexus Card / Cod Promo: \033[0m";
    std::cin >> codPromo;

    double procentReducere = 0.0;
    if (codPromo == "TECH10") procentReducere = 10.0;
    else if (codPromo == "STUDENT20") procentReducere = 20.0;
    else if (codPromo == "NEXUS30") procentReducere = 30.0;

    double pretTotalStandard = p.getPret() * cantitate;
    double valoareReducere = (pretTotalStandard * procentReducere) / 100.0;
    double pretFinal = pretTotalStandard - valoareReducere;

    try {
        const_cast<Depozit*>(this)->actualizeaza(id, -cantitate);
        // Desenare bon fiscal albastru/alb...
    } catch(const std::exception& e) {
        std::cout << "\033[31m[EROARE] " << e.what() << "\033[0m\n";
    }
}
```

6. Concluzii și Sustenabilitate

Arhitectura implementată pentru **"Nexus IT Hub"** demonstrează scalabilitate ridicată și stabilitate sub stres. Prin eliminarea elementelor grafice instabile și adoptarea encodării ANSI native, programul rulează fluid în orice consolă. Proiectul atinge toate obiectivele academice stabilite pentru disciplina Programare Orientată pe Obiecte, oferind un model practic complet gata de utilizare comercială sau audit managerial.